Data

	$X_1 \ldots n_1$	$Y_1 \ldots n_2$	$Z_1 \ldots n_3$
Taxa 1	$x_{11} \ldots x_{1n_1}$	$y_{11} \ldots y_{1n_2}$	
Taxa 2			

Notes:

- For now, assume $n_1 = n_2 = n_3 = n$
- In vineyard data, we have <u>some</u> correspondence, i.e.

  x_{11}, x_{12}, x_{13} = samples A, B, C in time 1

  y_{11}, y_{12}, y_{13} = same

  → not using this fact for now

Standardization

→ Possible data <u>cleaning</u> step is to standardize down <u>columns</u>, if they are very different scales

→ In vineyard data and simulated data, columns are repeat measures over time, so no need to stdize.

→

VSAT Procedure

① Standardize by row (taxa), within each group. This is for future computational convenience and clarity, since we are interested in pairwise correlations.

Define:

$$u_{ij} = \frac{x_{ij} - \overline{x_i}}{sd(x_i)}$$

$$v_{ij} = \frac{y_{ij} - \overline{y_i}}{sd(y_i)}$$

$$w_{ij} = \frac{z_{ij} - \overline{z_i}}{sd(z_i)}$$

② Choose an initial set

$$A \subseteq \{1, \ldots, m\}$$

where m = number of rows/taxa

→

③ Define the "connection" between a ~~point~~ taxa $\{i\} \notin A$ and A to be the average correlation between pairs $i, k : k \in A$.

$$S_1(i, A) := \frac{1}{|A|} \sum_{k \in A} \mathrm{cor}(\vec{x}_i, \vec{x}_k)$$

$$= \frac{1}{|A|} \sum_{k \in A} \frac{1}{n_1} \sum_{j=1}^{n_1} u_{ij} u_{kj}$$

$$= \frac{1}{n_1} \sum_{j=1}^{n_1} u_{ij} \left(\frac{1}{|A|} \sum_{k \in A} u_{kj} \right)$$

$$= \frac{1}{n_1} \sum_{j=1}^{n_1} u_{ij} \, m_{1j}^{(A)}$$

where $m_{1j}^{(A)} := \frac{1}{|A|} \sum_{k \in A} u_{kj}$,

i.e. the mean of the jth column across rows in A.

so $$S_1(i, A) = \mathrm{cov}(\vec{u}_i, \vec{m}_1^{(A)})$$

Note:

- Cov as a shortcut works here because $\vec{u}_i$ and $\vec{m}_1^{(A)}$ are both centered at 0.

- However, it is <u>not</u> equivalent to $\mathrm{Cor}(\vec{u}_i, \vec{m}_1^{(A)})$ because $\vec{m}_1^{(A)}$ does not have std dev of one.

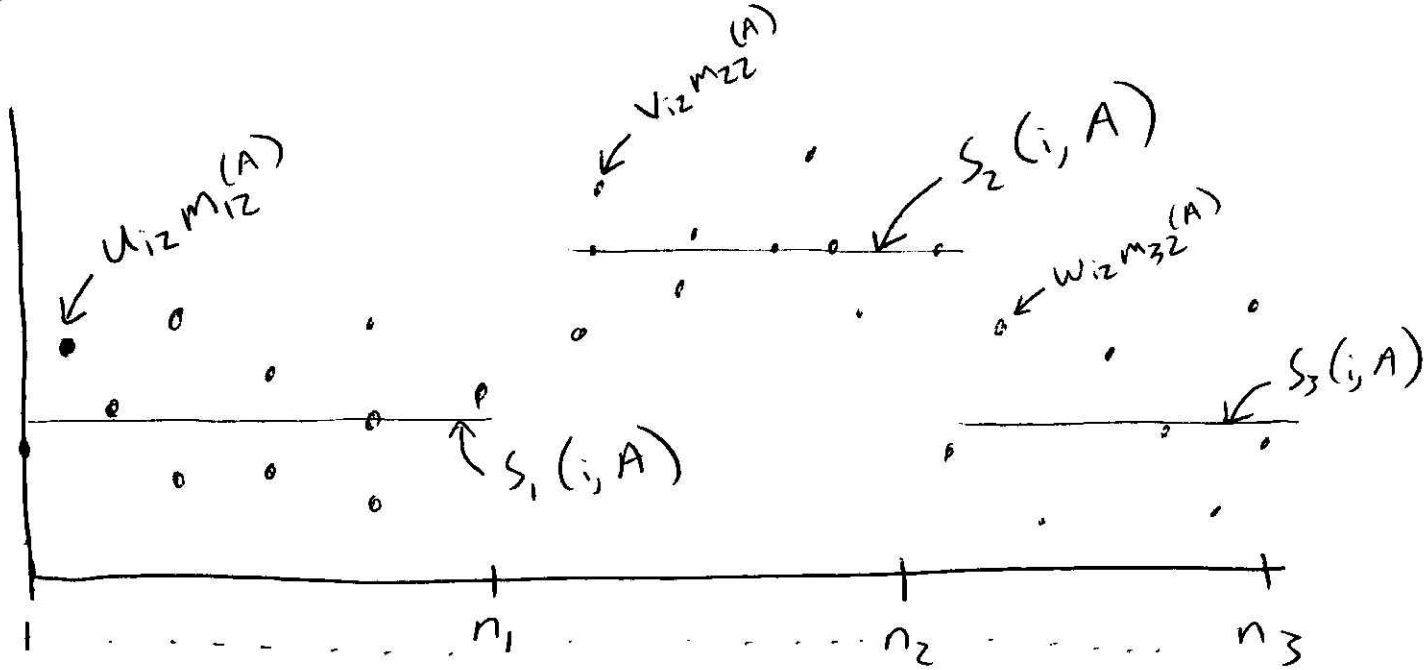
(4) The "contributions" to the metric $S_1(i, A)$ are therefore the individual "pieces" being averaged:

$$\{ u_{i1} m_{11}^{(A)}, u_{i2} m_{12}^{(A)}, \ldots, u_{in_1} m_{1n_1}^{(A)} \}$$

→ Similarly for $S_2(i, A)$ and $S_3(i, A)$, which have contribs.

$$\{ v_{ij} m_{2j}^{(A)} \} \text{ and } \{ w_{ij} m_{3j}^{(A)} \}$$

(5) We thus have an ANOVA-esque situation:



Two caveats:

- These "contributions" are <u>not</u> independent. The dependence is introduced by the standardizing

$$u_{ij} = \frac{x_{ij} - \overline{x_i}}{sd(x_i)}$$

involve other j's

- These contributions may be extremely non-Normal; in fact, we see very skewed distributions in simulated data.

(6) [Parametric version]

For simplicity, let's define

$$\tilde{u}_{ij} := u_{ij}\, m_{ij}^{(A)}, \quad \text{for some}$$

fixed i and A.

$\tilde{v}_j$, $\tilde{w}_j$ similarly.

Then, the parametric model is

$$\begin{aligned} SS_{\text{Treatments}} &= \left[S_1(i,A) - \bar{S}(i,A)\right]^2 \\ &+ \left[S_2(i,A) - \bar{S}(i,A)\right]^2 \\ &+ \left[S_3(i,A) - \bar{S}(i,A)\right]^2 \end{aligned}$$

where $\bar{S}(i,A) = \frac{1}{3}\sum_{t=1}^{3} S_t(i,A)$

$$\begin{aligned} SS_{\text{error}} &= \sum_{j=1}^{n_1} \left(\tilde{u}_j - S_1(i,A)\right)^2 \\ &+ \sum_{j=1}^{n_2} \left(\tilde{v}_j - S_2(i,A)\right)^2 \\ &+ \sum_{j=1}^{n_3} \left(\tilde{w}_j - S_3(i,A)\right)^2 \end{aligned}$$

so,

$$F = \frac{SS_{Treatment} / (3-1)}{SS_{Error} / (\underbrace{n_1+n_2+n_3}_{n_T} - 3)}$$

$$\approx F_{2,\, n_T-3}$$

<u>if</u>

- $\{\tilde{u}_i\}$ are (approx) independent
- $\{\tilde{u}_i\}$ are (approx) Normal
- $\{\tilde{u}_i\}$ has the same variance as $\{\tilde{v}_i\}$ and $\{\tilde{w}_i\}$

→ First one may be okay, we are going to ignore that math!

→ Second one is probably <u>not</u> okay and may explain our weird p-vals

→ Third one is maybe okay - def. fine in simulated data, but possibly not in vineyard data.

(6) [Non-parametric version]

Kruskall-Wallis test:

- uses <u>ranks</u> of the $\{\tilde{u}_j, \tilde{v}_j, \tilde{w}_j\}$ values
- does <u>not</u> require Normality
- <u>does</u> still require independence, but we will still ignore this issue :)
- does <u>not</u> require equal variance

Notes:

- No reason to code this from scratch. Use

  kruskal.test(x, g) where

  $$x = c(\tilde{u}_j, \tilde{v}_j, \tilde{w}_j) \text{ and}$$

  $$g = c(\underbrace{1\ldots1}_{n_1}, \underbrace{2\ldots2}_{n_2}, \underbrace{3\ldots3}_{n_3})$$

- This may move slow for large data, but I think not...